Inteligência Artificial no Ensino dos Fundamentos de Programação Competitiva: Recuperação, Validação e Limitações sob Orçamento Zero de API

Autoria a preencher[1]

[1]Instituição e departamento a preencher
Cidade – Estado – Brasil

email@instituicao.br

Abstract. *This paper presents AutoTutor as an exploratory prototype for learning competitive programming foundations. It combines language models, retrieval-augmented generation, persistent learning artifacts, and executable verification. We map implemented mechanisms to AutoCode and OI-Assistant, with Codigen as an additional reference. A retrospective analysis of four foundational tasks exposes interpretation, output-format and truncation failures in a lightweight local model. A separate API pilot verifies connectivity and validation infrastructure but does not evaluate beginner learning outcomes. We propose a controlled comparison of retrieval and generation strategies using Gemini 3.1 Flash-Lite within the user's declared free quota. The contribution is an auditable architecture and evaluation protocol, without claims of educational effectiveness, general model superiority or fine-tuning.*

Resumo. *Este artigo apresenta o AutoTutor como protótipo exploratório para aprendizagem dos fundamentos de programação competitiva. A aplicação combina modelos de linguagem, geração aumentada por recuperação, persistência de materiais e verificação executável. Mapeiam-se mecanismos implementados aos trabalhos AutoCode e OI-Assistant, com Codigen como referência adicional. Uma análise retrospectiva de quatro tarefas básicas expõe falhas de interpretação, formato e truncamento de um modelo local leve. Um piloto separado de API verifica conectividade e infraestrutura de validação, sem avaliar aprendizagem de iniciantes. Propõe-se comparação controlada de estratégias com Gemini 3.1 Flash-Lite na cota gratuita declarada pelo usuário. A contribuição é uma arquitetura auditável e um protocolo de avaliação, sem alegações de eficácia educacional, superioridade geral ou ajuste de pesos.*

1. Introdução e delimitação

A formação inicial em programação competitiva envolve interpretar enunciados, percorrer vetores, distinguir casos, estimar complexidade e relacionar uma solução ao resultado esperado. Um iniciante pode compreender uma explicação aparentemente convincente e ainda construir um índice incorreto ou confundir a ordem de visitação de um grafo. Assistentes de IA podem oferecer exemplos e dicas, mas a fluência do texto não assegura a correção do procedimento.

O AutoTutor reúne estudo, biblioteca, problemas, juiz local e acompanhamento em uma aplicação Python/Streamlit com persistência SQLite. O presente recorte prioriza fundamentos e simulação Ad Hoc, vetores e contagem, ordenação, somas de prefixos, busca binária, guloso introdutório, BFS, DFS e programação dinâmica elementar. Árvores de segmentos, propagação preguiçosa e Block-Cut Tree ficam fora do conjunto principal de avaliação. A existência de recursos avançados na aplicação não define o público nem a contribuição deste trabalho.

A pergunta orientadora é: *como recuperação de referências e validação independente podem apoiar a produção de explicações e exercícios introdutórios, considerando limitações de modelos locais e uma API sem orçamento financeiro?* As contribuições são: (i) mapear os mecanismos efetivamente implementados e sua relação com a literatura; (ii) analisar falhas locais de forma rastreável; e (iii) estabelecer um protocolo viável de avaliação com orçamento zero. Este texto relata engenharia e evidências exploratórias. Não apresenta um experimento concluído com estudantes.

2. Fundamentação de inteligência artificial

2.1. Modelos de linguagem, inferência e recursos

Um modelo autorregressivo descreve a distribuição de uma resposta y condicionada ao contexto x por

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}). \tag{1}$$

Os parâmetros θ decorrem do treinamento. O AutoTutor realiza inferência com modelos existentes, sem alterar seus pesos. Salvar um estudo ou adicionar documentos ao banco não constitui treinamento, aprendizagem contínua ou *fine-tuning*. O Transformer fundamenta essa família de modelos pelo mecanismo de atenção, que relaciona representações de posições do contexto [Vaswani et al. 2017]. A aplicação utiliza o serviço de inferência e não implementa a arquitetura interna do provedor.

O Llama 3.2 1B permanece como opção local por restrição de recursos. Um modelo compacto pode operar na máquina disponível, mas sua capacidade pedagógica precisa ser medida. O custo de memória inclui pesos, contexto, estruturas intermediárias e processo de execução. Assim, tamanho do arquivo e número de parâmetros não descrevem isoladamente a RAM necessária. A configuração local limita threads e solicita descarregamento após a resposta. Esses controles afetam a operação, sem certificar correção.

O acesso por API transfere a inferência para outra infraestrutura. Pode reduzir pressão de memória no computador, mas acrescenta dependência de rede, cota, indisponibilidade e políticas do provedor. Não é possível deduzir a superioridade de um modelo apenas de sua localização. Temperatura reduzida e saída JSON também não garantem verdade: regulam variabilidade e forma, respectivamente.

2.2. RAG, conhecimento externo e proveniência

RAG condiciona a geração a evidências recuperadas de um acervo [Lewis et al. 2020]. No protótipo, o fluxo pode ser descrito como

$$D_k = \mathrm{Retrieve}(q, \mathcal{D}), \qquad y \sim p_\theta(\cdot \mid q, D_k, s), \tag{2}$$

em que q representa o pedido, $\mathcal{D}$ o corpus, D_k os trechos selecionados e s as instruções. O recuperador e o gerador não são treinados conjuntamente. O armazenamento expande a memória externa disponível na inferência.

Há falhas distintas de cobertura, recuperação e utilização. Um documento pode faltar no corpus, existir sem aparecer entre os primeiros resultados ou ser recuperado e interpretado incorretamente. URLs no rodapé não demonstram que o conteúdo foi utilizado. Por isso, o sistema diferencia texto integral externo, material autoral e referência sem texto, registrando origem e trechos recuperados. Materiais gerados por IA permanecem disponíveis para estudo posterior, mas não substituem automaticamente fontes externas na recuperação experimental.

BM25 combina evidência lexical, saturação de frequência e normalização de comprimento [Robertson and Zaragoza 2009]. É econômico e auditável, mas pode não recuperar paráfrases com vocabulário distinto. Embeddings, motivados por representações como Sentence-BERT [Reimers and Gurevych 2019], aproximam textos por vetores. A similaridade cosseno $u^\top v/(\|u\|\|v\|)$ mede proximidade representacional, sem provar equivalência algorítmica.

O experimento implementa recuperação lexical, densa, híbrida, reordenação leve, diversificação e expansão por competências. A fusão RRF soma $1/(60 + \mathrm{pos}(d))$ sobre listas, com posições iniciando em um nesta versão [Cormack et al. 2009]. O re-ranking atual usa evidência lexical e metadados, sem um modelo neural de relevância. A diversificação usa uma aproximação por sobreposição de palavras. Nenhuma dessas opções demonstrou superioridade no recorte introdutório. O objetivo inicial é avaliar se melhoram evidências úteis, antes de adotar infraestrutura mais complexa.

2.3. Competências e evidências do estudante

Uma competência representa um objetivo observável, como construir prefixos sem erro de índice ou usar uma fila para calcular distâncias sem pesos. Um grafo editorial relaciona 18 competências a pré-requisitos recomendados e contratos. Essas relações orientam consulta e especificação, sem estabelecer uma única ordem obrigatória de aprendizagem. A revisão por professores permanece pendente.

O protótipo BKT mantém uma probabilidade latente de conhecimento com parâmetros de conhecimento inicial, aprendizagem, acerto por acaso e erro apesar do domínio [Corbett and Anderson 1994]. Para resposta correta, uma atualização simplificada é

$$P(L \mid C) = \frac{P(L)(1 - P(S))}{P(L)(1 - P(S)) + (1 - P(L))P(G)}, \tag{3}$$

seguida da transição $P(L') = P(L \mid C) + (1 - P(L \mid C))P(T)$. Aqui os parâmetros são hipotéticos e não foram ajustados a estudantes. Repetir tentativas do mesmo item não multiplica evidência independente. Menos de três itens distintos permanece insuficiente, e esse mínimo é uma decisão editorial, sem validade diagnóstica demonstrada.

Objetivos graduais, dicas e feedback motivam o desenho pedagógico, mas são diferentes de uma técnica de IA e de uma medida de aprendizagem. Um veredito WA pode resultar de índice, interpretação ou implementação. O modelo não deve convertê-lo automaticamente em ausência de conhecimento. Contagens e taxas devem ser calculadas por

software e fornecidas ao modelo como evidências; gerar uma interpretação não autoriza inventar estatísticas.

3. Trabalhos relacionados e mecanismos aproveitados

O AutoCode propõe ciclos de geração e validação de problemas e testes [Zhou et al. 2026]. O AutoTutor aproveita a ideia de confrontar soluções e procurar inconsistências, mas restringe o experimento a contratos conhecidos de entrada e saída. São implementados compilação isolada, exemplos e comparação com oráculos e referências confiáveis. Faltam um benchmark de vereditos oficiais, revisão de originalidade e avaliação equivalente à do artigo. A consistência próxima de 99% relatada no AutoCode pertence à avaliação daquele trabalho e não corresponde à eficácia educacional nem à acurácia deste protótipo.

O OI-Assistant integra recuperação de problemas semelhantes e aprendizagem interativa [Su et al. 2025]. O AutoTutor distingue recomendação de problemas e recuperação de materiais: a primeira utiliza representações lexicais e conceituais; a segunda usa a biblioteca externa e contexto com proveniência. Recuperação por competências é uma extensão experimental local, sem alegar reprodução integral do OI-Assistant.

Codigen relaciona autoria e personalização com modelos de linguagem [Ez-zaouia and Boumarafi 2025]. É uma referência adicional para a edição do pedido, profundidade do estudo e apresentação do conteúdo. A aplicação não reproduz sua interface de autoria visual nem seus resultados. A contribuição do AutoTutor é integrar e tornar observáveis mecanismos, não apresentar um novo modelo fundacional.

4. Arquitetura, estado atual e melhorias propostas

A aplicação persiste usuários, turmas, materiais, trilhas e submissões em banco local. Páginas próprias permitem retomar estudos, anotações e referências. O acompanhamento do professor e a proposta de trios são recursos da plataforma, fora das variáveis centrais desta avaliação. Sua existência não demonstra que a análise automática esteja correta.

O corpus contém 5551 registros: 302 textos externos, 19 conteúdos autorais e 5230 referências. Há 6837 chunks no banco, incluindo diferentes categorias. Portanto, a quantidade de referências não deve ser apresentada como quantidade de artigos integralmente lidos. Os textos externos incluem 164 páginas de CP-Algorithms e 138 de USACO Guide, com licenças e origem. Não se afirma importação integral de todos os sites solicitados [CP-Algorithms contributors 2026, CP Initiative 2026].

Uma auditoria adicional de nove consultas básicas no BM25 identificou uma falha de vocabulário: “Vetores e contagem” não retornava texto externo. A inclusão de equivalências português–inglês passou a recuperar candidatos, sem demonstrar relevância pedagógica. Outras consultas retornaram tópicos como matroides, 0–1 BFS e DP de perfil entre os primeiros resultados. Essa inspeção de fontes evidencia necessidade de curadoria por nível; não substitui julgamentos completos de relevância nem autoriza calcular precisão e recall.

O banco local possui 39 contratos executáveis. Nesta revisão, nove objetivos básicos receberam associação explícita a contratos; isso não significa que todos os 39 sejam introdutórios. Soma de prefixos passou a ser o padrão do piloto, e o painel experimental

oculta tópicos avançados até seleção explícita. Um teste offline do contrato de prefixos passou pela validação, utilizando solução conhecida. Ele verifica integração, não qualidade de geração por modelo.

O caminho experimental segue: objetivo e preferências; contrato e competências; recuperação; geração estruturada; schema; compilação e testes; armazenamento com fontes e validação. O juiz usa namespaces Linux, rede ausente, filesystem restrito e limites de tempo, memória, saída e processos. A ausência de isolamento bloqueia execução. Trata-se de protótipo local, sem auditoria para exposição pública multiusuário.

As próximas melhorias prioritárias são curar fontes introdutórias por objetivo, reduzir recuperação de conteúdo avançado irrelevante, verificar exemplos com ferramentas determinísticas e avaliar dicas por rubrica humana. O índice semântico e o grafo devem justificar seu custo por resultados mensuráveis. Cache local de materiais já validados pode economizar chamadas, preservando autoria, modelo e versão; o cache de respostas por chave de conteúdo permanece proposta, não implementação atual.

5. Evidências observadas e limites dos modelos locais

5.1. Reanálise de tarefas básicas com Llama 1B

Uma avaliação anterior registrou cinco tarefas sintéticas com Llama 3.2 1B e limite de 256 tokens de saída. Para o novo recorte, considera-se apenas BFS, DFS, DP e prefixos; o item avançado fica excluído. As respostas recebiam referências externas. A Tabela 1 distingue modos de falha e evita reduzir todos a erro aritmético.

Tabela 1. Quatro tarefas básicas da avaliação anterior. Tempos observados não isolam carga concorrente.

Tema	Tempo (s)	Evidência registrada
BFS	24,82	Esperava-se a sequência de distâncias $[0, 1, 1, 2]$. A saída forneceu pares diferentes e confundiu a descrição da busca.
DFS	17,85	Esperava-se 4; a resposta forneceu resultado 1 e rótulos de fontes em formato inválido.
DP	34,04	A saída truncou durante uma enumeração, impedindo leitura do JSON. Não isola falha de raciocínio.
Prefixos	26,38	O valor 5 aparece, mas a justificativa apresenta fórmula/resultado contraditórios e o campo esperado não é respeitado.

O verificador original não aprovou os quatro itens. Isso mede conformidade com o contrato e consistência da resposta nessas condições, não uma acurácia geral de 0% em programação. O limite curto pode explicar o truncamento de DP. Para prefixos, um número coincidente não corrige a justificativa nem o schema. A ausência de repetições e de controle de carga impede inferência estatística sobre o modelo.

O Qwen2.5 3B também foi testado anteriormente em perfis sintéticos. Houve geração de material que excedeu 300 segundos, inversão de taxas e justificativas com informações não fornecidas. A configuração e as tarefas diferem das do Llama. Portanto, não se constitui ranking entre famílias. O caso evidencia que JSON válido e IDs sem repetição não asseguram fidelidade a dados. Um teste de conectividade do Llama respondeu

“OK” em 3,81 segundos com tamanho de modelo reportado de 1466852966 bytes; esse valor não é medição de RAM total da aplicação.

5.2. O que o piloto de API permite afirmar

O adaptador Gemini 3.1 Flash-Lite recebeu resposta em quatro chamadas reais anteriores, incluindo conexão, somando 5744 tokens retornados pelo provedor. O piloto de geração utilizou um contrato avançado e não compõe a avaliação de fundamentos. Seus artefatos ficam preservados como evidência de infraestrutura. Não foi executada nesta revisão uma comparação pareada Gemini–Llama em problemas básicos. Consequentemente, a API está operacional, mas sua vantagem pedagógica no público-alvo permanece hipótese.

6. Gratuidade e protocolo de avaliação proposto

A documentação consultada em 5 de novembro de 2026 lista entrada e saída do Gemini 3.1 Flash-Lite sem cobrança na modalidade gratuita. Cotas dependem do projeto e devem ser consultadas no AI Studio [Google 2026b]. A aplicação não identifica o plano de faturamento pelo texto de uma resposta. O usuário declarou utilizar cota gratuita, e não há autorização para despesa. O adaptador não habilita faturamento nem usa fallback pago.

O limite de seis tentativas por dia UTC é uma proteção local do projeto, não a cota publicada pelo Google. Também há teto de tokens reservados, entrada limitada, intervalo mínimo e interrupção após erros de cota. Não se deve confundir o reinício UTC do ledger com o reinício de cota do provedor. Falhas continuam no denominador e consomem reservas. Grounding por busca e cache gerenciado não são necessários: o RAG usa SQLite e embeddings locais, sem depender desses serviços.

Os termos de serviços gratuitos permitem uso de entradas e saídas para melhoria de produtos e revisão. Assim, esta fase utiliza somente dados públicos e sintéticos, sem nomes, códigos privados ou históricos identificáveis de estudantes [Google 2026a]. Uma implantação educacional real exige revisar os termos aplicáveis e o tratamento de dados antes de enviar informações de participantes.

Propõe-se um primeiro piloto com quatro temas (simulação, prefixos, BFS e DP introdutória) e três condições: contrato sem recuperação, contrato com BM25 e contrato com BM25 mais especificação pedagógica. São 12 chamadas sem reparo. Mantendo o teto de seis, seriam necessários pelo menos dois dias com a capacidade diária integral disponível, condicionados às cotas reais. Repetir cada célula três vezes demanda 36 chamadas e pelo menos seis dias nessas condições. Esses números são planejamento de chamadas, não promessa de disponibilidade gratuita ou resultado executado.

O piloto compara pacotes de contexto. Para isolar a contribuição da especificação, uma ablação posterior manterá o contexto recuperado idêntico. Ordem aleatória, tarefas pareadas, mesmo modelo, temperatura, orçamento de saída e critérios reduzem confundimento. Um comparativo local/API adicional deve padronizar prompts, schema, limite de saída e timeout, registrar versões e separar falhas de formato, raciocínio, truncamento e execução. A configuração local antiga não oferece esse controle.

As métricas técnicas incluem schema, compilação, exemplos e testes independentes. A rubrica humana cobre clareza, ambiguidade, necessidade da competência, coerência das restrições, progressão e dicas. Precision, Recall, MRR e nDCG exigem

julgamentos de relevância revisados; não foram calculados sem rótulos. Dois revisores independentes devem avaliar saídas sem conhecer a condição. Taxas devem indicar denominador e incerteza. Ganho de aprendizagem exige estudo separado, com instrumentos e participantes adequados.

7. Ameaças à validade e conclusão

As tarefas locais são poucas e sintéticas, com limite de saída curto e sem repetições. Dados de API disponíveis referem-se a outro contrato. O corpus tem cobertura desigual e inclui materiais avançados. O grafo e o BKT são editoriais, sem validação especializada ou calibração. Testes finitos não provam correção nem certificam qualidade pedagógica, e contratos conhecidos limitam originalidade algorítmica.

O AutoTutor constitui uma base funcional para investigar apoio aos fundamentos: fontes com proveniência, geração configurável, persistência e validação executável estão disponíveis. A análise local mostra por que modelos leves precisam de verificações, inclusive em tarefas simples. A cota gratuita permite planejar experimentos pequenos e progressivos, sem garantir escala de turma ou reproduzir resultados de sistemas de maior capacidade. A próxima contribuição científica depende de executar o protocolo básico e revisar suas saídas, preservando a distinção entre implementação, evidência técnica e aprendizagem.

Referências

Corbett, A. T. and Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4:253–278. DOI: 10.1007/BF01099821.

Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, pages 758–759. DOI: 10.1145/1571941.1572114.

CP-Algorithms contributors (2026). Algorithms for competitive programming. `https://cp-algorithms.com/`. Revisão bbbbb6f04e5c9a665c8477ec68ee4f28b1367dd7. Licença CC BY-SA 4.0.

CP Initiative (2026). Usaco guide. `https://usaco.guide/`. Repositório cpinitiative/usaco-guide, revisão a2a25966a3c8e6cb7e0a87985b0abaac99e2ad40. Licença CC BY-NC-SA 4.0.

Ez-zaouia, M. and Boumarafi, Y. (2025). Codigen: Enhancing personalized programming education with visual authoring and llms. In *IEEE Symposium on Visual Languages and Human-Centric Computing*. DOI: 10.1109/VL-HCC65237.2025.00038.

Google (2026a). Gemini api additional terms of service. `https://ai.google.dev/gemini-api/terms`. Consulta em 5 nov. 2026.

Google (2026b). Gemini developer api pricing; rate limits. `https://ai.google.dev/gemini-api/docs/pricing`. Limites: `https://ai.google.dev/gemini-api/docs/rate-limits`. Consulta em 5 nov. 2026.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented

generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. arXiv:2005.11401.

Reimers, N. and Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992. DOI: 10.18653/v1/D19-1410.

Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389. DOI: 10.1561/1500000019.

Su, Y., Nie, P., and Meng, X. (2025). Oi-assistant: A retrieval augmented system for similar problem discovery and interactive learning in competitive programming. *Olympiads in Informatics*, 19:129–144. DOI: 10.15388/ioi.2025.09.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. arXiv:1706.03762.

Zhou, S., Zheng, Z., Liu, K., Shen, Z., Cheng, Z., et al. (2026). Autocode: Llms as problem setters for competitive programming. In *International Conference on Learning Representations*. arXiv:2510.12803.